
Data Structures

BS (CS) _Fall_2025

Lab_06 Task



Learning Objectives:

1. Linked Lists (Doubly, Circular)

Lab Task 1: Train Coach Management

Scenario

You are developing a railway management system that needs to handle train composition. Your task is to implement the system using Doubly Linked Lists to allow to add add/remove coaches dynamically.

Task Requirements

Create a doubly linked list where each node contains:

- Coach Number (string, e.g., "C001", "C002")
- Passenger Capacity (integer)
- Current Occupancy (integer, cannot exceed capacity)
- Coach Type (string: "Economy", "Business", "Dining", "Sleeper")
- Pointer to next coach
- Pointer to previous coach

Core Functionalities to Implement

1. Train Composition Management:

- `addCoachToFront(coachNum, capacity, type)` - Add coach at train front (engine side)
- `addCoachToRear(coachNum, capacity, type)` - Add coach at train rear
- `removeCoachFromFront()` - Remove front coach
- `removeCoachFromRear()` - Remove rear coach
- `removeCoach(coachNumber)` - Remove specific coach by number
- `displayTrainComposition()` - Show complete train layout

2. Coach Operations:

- `rearrangeCoach(coachNum, newPosition)` - Move coach to different position
- `findCoach(coachNumber)` - Find coach and display its neighbors
- `swapCoaches(coach1, coach2)` - Swap positions of two coaches
- `setOccupancy(coachNum, passengers)` - Update passenger count

3. Advanced Operations:

- `calculateTotalCapacity()` - Sum of all coach capacities

- calculateCurrentOccupancy() - Sum of all current passengers

4. Implementation Guidelines

- Implement a trainCoach class with appropriate member functions
- Use proper memory management (constructor, destructor, copy constructor)

Test cases:

```
//1. Add coach to front and rear (check presence)
TEST(TrainCoachTest, AddFrontRear) {
    TrainCoach t;
    t.addCoachToFront("C002", 60, "Economy");
    t.addCoachToFront("C001", 50, "Business"); // now C001 front then C002
    t.addCoachToRear("C003", 30, "Dining");
    EXPECT_TRUE(t.hasCoach("C001"));
    EXPECT_TRUE(t.hasCoach("C002"));
    EXPECT_TRUE(t.hasCoach("C003"));
}

// 2. Display composition (just call print function)
TEST(TrainCoachTest, DisplayComposition) {
    TrainCoach t;
    t.addCoachToRear("C001", 40, "Sleeper");
    t.addCoachToRear("C002", 60, "Economy");
    t.displayTrainComposition(); // student should visually verify order
}

// 3. Remove front and rear
TEST(TrainCoachTest, RemoveFrontRear) {
    TrainCoach t;
    t.addCoachToRear("C001", 40, "Sleeper");
    t.addCoachToRear("C002", 50, "Economy");
    EXPECT_TRUE(t.removeCoachFromFront());
    EXPECT_TRUE(t.hasCoach("C002"));
    EXPECT_FALSE(t.hasCoach("C001"));
    EXPECT_TRUE(t.removeCoachFromRear());
    EXPECT_FALSE(t.hasCoach("C002"));
}

// 4. Remove specific coach by number
TEST(TrainCoachTest, RemoveSpecific) {
    TrainCoach t;
    t.addCoachToRear("C010", 45, "Business");
    t.addCoachToRear("C011", 35, "Dining");
```

```

    EXPECT_TRUE(t.removeCoach("C010"));
    EXPECT_FALSE(t.hasCoach("C010"));
    EXPECT_TRUE(t.hasCoach("C011"));
}

// 5. setOccupancy success and failure (exceed capacity)
TEST(TrainCoachTest, SetOccupancy) {
    TrainCoach t;
    t.addCoachToRear("C100", 100, "Economy");
    EXPECT_TRUE(t.setOccupancy("C100", 50)); // OK
    EXPECT_FALSE(t.setOccupancy("C100", 200)); // exceeds capacity
    EXPECT_FALSE(t.setOccupancy("NOT", 10)); // coach not exists
}

// 6. calculate totals (capacity & occupancy)
TEST(TrainCoachTest, Totals) {
    TrainCoach t;
    t.addCoachToRear("C1", 30, "Economy");
    t.addCoachToRear("C2", 40, "Business");
    t.setOccupancy("C1", 10);
    t.setOccupancy("C2", 20);
    EXPECT_EQ(t.calculateTotalCapacity(), 70);
    EXPECT_EQ(t.calculateCurrentOccupancy(), 30);
}

// 7. rearrange coach to front, middle and end
TEST(TrainCoachTest, RearrangeCoach) {
    TrainCoach t;
    t.addCoachToRear("A", 10, "Economy");
    t.addCoachToRear("B", 10, "Economy");
    t.addCoachToRear("C", 10, "Economy");
    // Move C to front (pos 1)
    EXPECT_TRUE(t.rearrangeCoach("C", 1));
    // Move A to position 3 (end)
    EXPECT_TRUE(t.rearrangeCoach("A", 3));
    // invalid position
    EXPECT_FALSE(t.rearrangeCoach("B", 0));
}

// 8. findCoach (prints neighbors) - call function
TEST(TrainCoachTest, FindCoach) {
    TrainCoach t;
    t.addCoachToRear("X1", 20, "Sleeper");
    t.addCoachToRear("X2", 25, "Dining");
    t.addCoachToRear("X3", 30, "Business");

```

```

    EXPECT_TRUE(t.findCoach("X2"));
    EXPECT_FALSE(t.findCoach("NOPE"));
}

// 9. swap coaches (swap contents approach)
TEST(TrainCoachTest, SwapCoaches) {
    TrainCoach t;
    t.addCoachToRear("S1", 10, "Economy");
    t.addCoachToRear("S2", 20, "Business");
    t.setOccupancy("S1", 5);
    t.setOccupancy("S2", 10);
    EXPECT_TRUE(t.swapCoaches("S1", "S2"));
    // After swap, S1 node will contain data of S2 originally and vice versa.
    EXPECT_TRUE(t.hasCoach("S1"));
    EXPECT_TRUE(t.hasCoach("S2"));
}

// 10. remove non-existing coach
TEST(TrainCoachTest, RemoveNonExisting) {
    TrainCoach t;
    t.addCoachToRear("AA", 10, "Economy");
    EXPECT_FALSE(t.removeCoach("ZZ")); // not present
    EXPECT_TRUE(t.hasCoach("AA"));    // existing unaffected
}

// 11. set occupancy negative
TEST(TrainCoachTest, SetOccupancyNegative) {
    TrainCoach t;
    t.addCoachToRear("N1", 10, "Economy");
    EXPECT_FALSE(t.setOccupancy("N1", -5)); // invalid negative
}

// 12. calculate totals empty train
TEST(TrainCoachTest, TotalsEmpty) {
    TrainCoach t;
    EXPECT_EQ(t.calculateTotalCapacity(), 0);
    EXPECT_EQ(t.calculateCurrentOccupancy(), 0);
}

// 13. sequence operations
TEST(TrainCoachTest, MultipleOperationsSequence) {
    TrainCoach t;
    t.addCoachToFront("F1", 20, "Business");
    t.addCoachToRear("R1", 30, "Economy");
    t.addCoachToRear("R2", 25, "Dining");

```

```

    t.setOccupancy("R1", 15);
    t.removeCoach("F1");
    EXPECT_FALSE(t.hasCoach("F1"));
    EXPECT_EQ(t.calculateCurrentOccupancy(), 15);
}

// 14. copy constructor deep copy test
TEST(TrainCoachTest, CopyConstructor) {
    TrainCoach t1;
    t1.addCoachToRear("C1", 10, "Economy");
    t1.setOccupancy("C1", 5);
    TrainCoach t2 = t1; // copy
    t2.setOccupancy("C1", 8); // modify copy
    // originals unchanged
    EXPECT_EQ(t1.calculateCurrentOccupancy(), 5);
    EXPECT_EQ(t2.calculateCurrentOccupancy(), 8);
}

// 15. assignment operator test
TEST(TrainCoachTest, AssignmentOperator) {
    TrainCoach a;
    a.addCoachToRear("AA", 10, "Economy");
    TrainCoach b;
    b.addCoachToRear("BB", 20, "Business");
    b = a; // assign
    EXPECT_TRUE(b.hasCoach("AA"));
    EXPECT_FALSE(b.hasCoach("BB"));
}

```

Lab Task 2: Circular Linked List Implementation

Scenario

You are tasked with implementing a fundamental circular linked list data structure from scratch. This implementation will serve as the foundation for more complex circular data structures used in applications like round-robin scheduling, circular buffers, and continuous media playback systems.

Task Requirements

Implement a Node and CircularLinkedList Class

Core Functionalities to Implement

- CircularLinkedList() - Default constructor to initialize empty list
- insert(data) - Insert data item at the end of circular linked list
 - If list is empty, new node becomes head and points to itself
 - If list has elements, insert after the last node and maintain circularity
- isEmpty() - Check if the circular linked list is empty
- print() - Display all elements starting from head until reaching head again

Search and Update Operations:

- search(data) - Search for specific element, return true if found, false otherwise
- update(oldData, newData) - Replace/update one data element with another
- insertAtIndex(index, data) - Insert new element at specified position (0-indexed)
 - Count elements in the list to determine valid positions
 - Handle edge cases for index 0 and end positions

Deletion Operations:

- deleteData(data) - Delete the first occurrence of specified data element
- Handle special cases: single node, head node deletion, last node deletion

Test Cases:

```
// 1. Empty list
TEST(CircularListTest, IsEmptyInitially) {
    CircularLinkedList cl;
    EXPECT_TRUE(cl.isEmpty());
    EXPECT_EQ(cl.size(), 0);
}

// 2. Insert first element
TEST(CircularListTest, InsertFirstElement) {
    CircularLinkedList cl;
    cl.insert(10);
    EXPECT_FALSE(cl.isEmpty());
}
```

```

    EXPECT_EQ(cl.size(), 1);
    EXPECT_EQ(cl.getAtIndex(0), 10);
}

// 3. Insert multiple elements
TEST(CircularListTest, InsertMultipleElements) {
    CircularLinkedList cl;
    cl.insert(1);
    cl.insert(2);
    cl.insert(3);
    EXPECT_EQ(cl.size(), 3);
    EXPECT_EQ(cl.getAtIndex(2), 3);
}

// 4. Search existing element
TEST(CircularListTest, SearchElement) {
    CircularLinkedList cl;
    cl.insert(5);
    cl.insert(15);
    cl.insert(25);
    EXPECT_TRUE(cl.search(15));
    EXPECT_FALSE(cl.search(99));
}

// 5. Update element
TEST(CircularListTest, UpdateElement) {
    CircularLinkedList cl;
    cl.insert(10);
    cl.insert(20);
    EXPECT_TRUE(cl.update(20, 25));
    EXPECT_TRUE(cl.search(25));
    EXPECT_FALSE(cl.update(99, 100));
}

// 6. Insert at index 0 (head)
TEST(CircularListTest, InsertAtHead) {
    CircularLinkedList cl;
    cl.insert(5);
    cl.insert(10);
    cl.insertAtIndex(0, 99);
    EXPECT_EQ(cl.getAtIndex(0), 99);
}

// 7. Insert at middle
TEST(CircularListTest, InsertAtMiddle) {

```



```

    CircularLinkedList cl;
    cl.insert(1);
    cl.insert(2);
    cl.insert(3);
    cl.insertAtIndex(1, 50);
    EXPECT_EQ(cl.getAtIndex(1), 50);
    EXPECT_EQ(cl.size(), 4);
}

// 8. Insert at end
TEST(CircularListTest, InsertAtEnd) {
    CircularLinkedList cl;
    cl.insert(10);
    cl.insert(20);
    cl.insertAtIndex(2, 30);
    EXPECT_EQ(cl.getAtIndex(2), 30);
}

// 9. Insert invalid index
TEST(CircularListTest, InsertInvalidIndex) {
    CircularLinkedList cl;
    cl.insert(1);
    EXPECT_FALSE(cl.insertAtIndex(5, 99));
}

// 10. Delete head element
TEST(CircularListTest, DeleteHead) {
    CircularLinkedList cl;
    cl.insert(10);
    cl.insert(20);
    cl.insert(30);
    EXPECT_TRUE(cl.deleteData(10));
    EXPECT_FALSE(cl.search(10));
}

// 11. Delete last element
TEST(CircularListTest, DeleteLast) {
    CircularLinkedList cl;
    cl.insert(10);
    cl.insert(20);
    cl.insert(30);
    EXPECT_TRUE(cl.deleteData(30));
    EXPECT_EQ(cl.size(), 2);
}

```

```
// 12. Delete middle element
TEST(CircularListTest, DeleteMiddle) {
    CircularLinkedList cl;
    cl.insert(1);
    cl.insert(2);
    cl.insert(3);
    EXPECT_TRUE(cl.deleteData(2));
    EXPECT_FALSE(cl.search(2));
}

// 13. Delete only element
TEST(CircularListTest, DeleteSingleNode) {
    CircularLinkedList cl;
    cl.insert(100);
    EXPECT_TRUE(cl.deleteData(100));
    EXPECT_TRUE(cl.isEmpty());
}

// 14. Delete non-existing element
TEST(CircularListTest, DeleteNonExisting) {
    CircularLinkedList cl;
    cl.insert(10);
    EXPECT_FALSE(cl.deleteData(999));
}

// 15. Print list (manual visual check)
TEST(CircularListTest, PrintList) {
    CircularLinkedList cl;
    cl.insert(1);
    cl.insert(2);
    cl.insert(3);
    cl.print(); // Just call (visual verification)
}
```